

Inwiefern beeinflusst eine limitierte Hardware-Ressource die Performance-Vorteile von GraphQL gegenüber REST-APIs unter steigender Systemlast

Antonio Keetman & Georg Sittnick

Wissenschaftliches Arbeiten SoSe 2026 Prof. Dr.

Hildebrand

Problem

GraphQL vermeidet Over- und Underfetching und gilt daher als netzwerkschonend. Der Preis: Jede Abfrage wird zur Laufzeit geparkt, validiert und per Resolver aufgelöst. In der Cloud fängt elastische Skalierung das ab aber auf Althardware konkurriert der Overhead direkt mit der Anwendungslogik.

Verwandte Arbeiten

Traditionelle **REST-APIs** nutzt standardisierte Endpunkte, führt jedoch zu komplexen Abfragen zu Over-fetching und unkoordinierten Waterfall-Requests [1, 2].

GraphQL löst diese Ineffizienz, indem es Client ermöglicht Datenstrukturen flexibel in einem einzigen Netzwerk-Roundtrip abzufragen [3, 4].

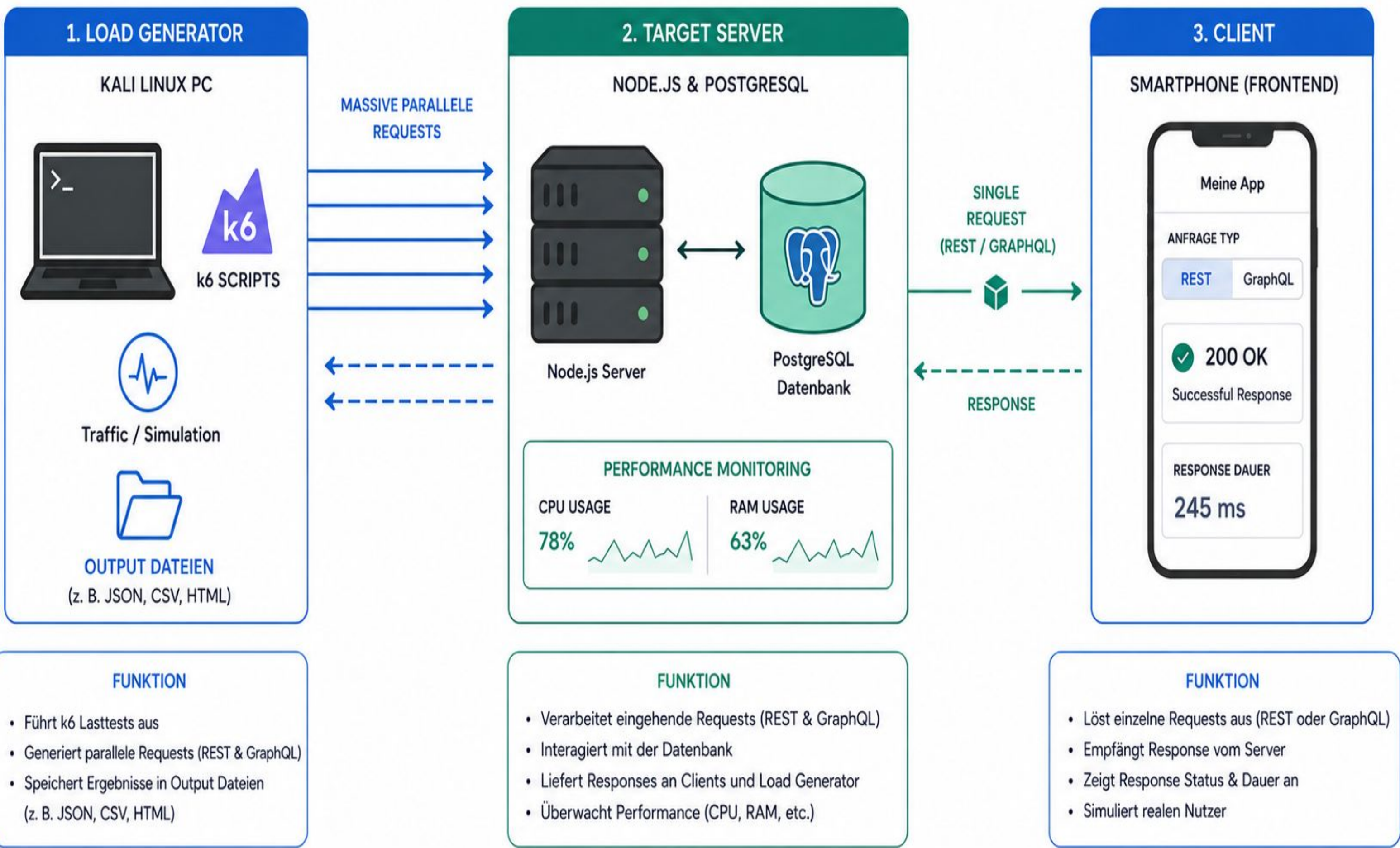
Studien belegen:

Hoher CPU-Overhead durch dynamisches Echtzeit-Parsing und Validierung auf dem Server. System kann nicht mehr skalieren bei hoher Parallelität (High-Concurrency) im Vergleich zu REST.

Ansatz

Kontrollierter End-to-End-Benchmark, das den Architekturpreis auf realer Althardware darstellt.
-> identische Codebasis und Datenbank.

- H1 GraphQL: Payload-Vorteil bei einfachen Listen
- H2 Tiefe Verschachtelung relativiert den Vorteil
- H3 GraphQL bindet mehr CPU & RAM am Server
- H4 Tiefe ohne Optimierung erzeugt N+1-Problem
- H5 GraphQL bricht unter Last früher ein
- H6 REST: stabilerer Durchsatz, robuster



Limitation & Ausblick

- Fehlende Optimierungs-Middleware erzeugt
- Worst-Case-Szenario.
- Lokales Netzwerk ignoriert reale WAN-Latenzen.
- Hardware-Ergebnisse schwer auf IoT übertragbar.
- Künftig Effekte von Optimierungsstrategien untersuchen.
- Physischen Stromverbrauch in Watt messen.

Referenzen

- [1] Björn-Hansen et al. (2020). Empirical study of GraphQL & client-side parsing. ACM SAC, 722–731.
- [2] Brito et al. (2019). Migrating to GraphQL: A practical assessment. IEEE SPLC, 114–123.
- [3] Brito et al. (2020). GraphQL vs. REST in serverless. ACM/IEEE ESEM.
- [4] Facebook Open Source (2015). GraphQL: A data query language. <https://graphql.org>.
- [5] Fielding, R. T. (2000). Architectural Styles & Network-based Software Architectures. PhD Thesis, UC Irvine.
- [6] Kern et al. (2018). Green software engineering: Nachhaltige Softwareentwicklung. Informatik Spektrum 41(1), 12–21.
- [7] Popescu & Radulescu (2023). Perf. evaluation of REST & GraphQL in mobile. CEEOL, 33–42.
- [8] Svensson & Johansson (2021). How Fast GraphQL is Compared to REST. Master's Thesis, KTH.
- [9] Witkowski & Skublewski-Paszkowska (2021). Comparison of REST & GraphQL. JCSI 18, 45–51.
- [10] Zimmermann & Stocker (2022). API composition: GraphQL federation vs. REST. Springer ICSSOC.

Ergebnisse

- GraphQL reduziert Payload bei abfragen von einfachen Listen stark.
- GraphQL belastet CPU und RAM stärker als REST.
- N+1-Problem erzeugt zehntausende Datenbankabfragen pro Request.
- GraphQL-Fehlerquote steigt unter Last stärker an.
- REST skaliert auf Althardware deutlich robuster.

